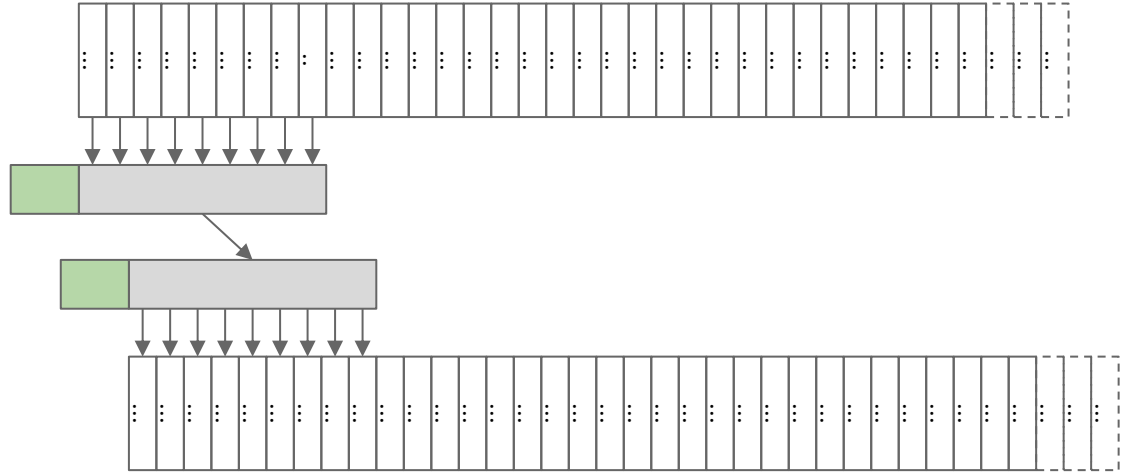




Lecture 11 (Transport 2)

TCP Implementation

Implementing TCP: byte Notation (Segments, Sequence Numbers)

Lecture 11, Spring 2026

Implementing TCP

- **byte Notation
(Segments, Sequence Numbers)**
- Maintaining State (Full Duplex,
Connection Setup and Teardown)
- Sliding Window
- Header

So far, we've used *packets* as the primary unit of data.

- Each packet has a number.
- Acks reference packet numbers.
- Window size expressed in terms of number of packets.

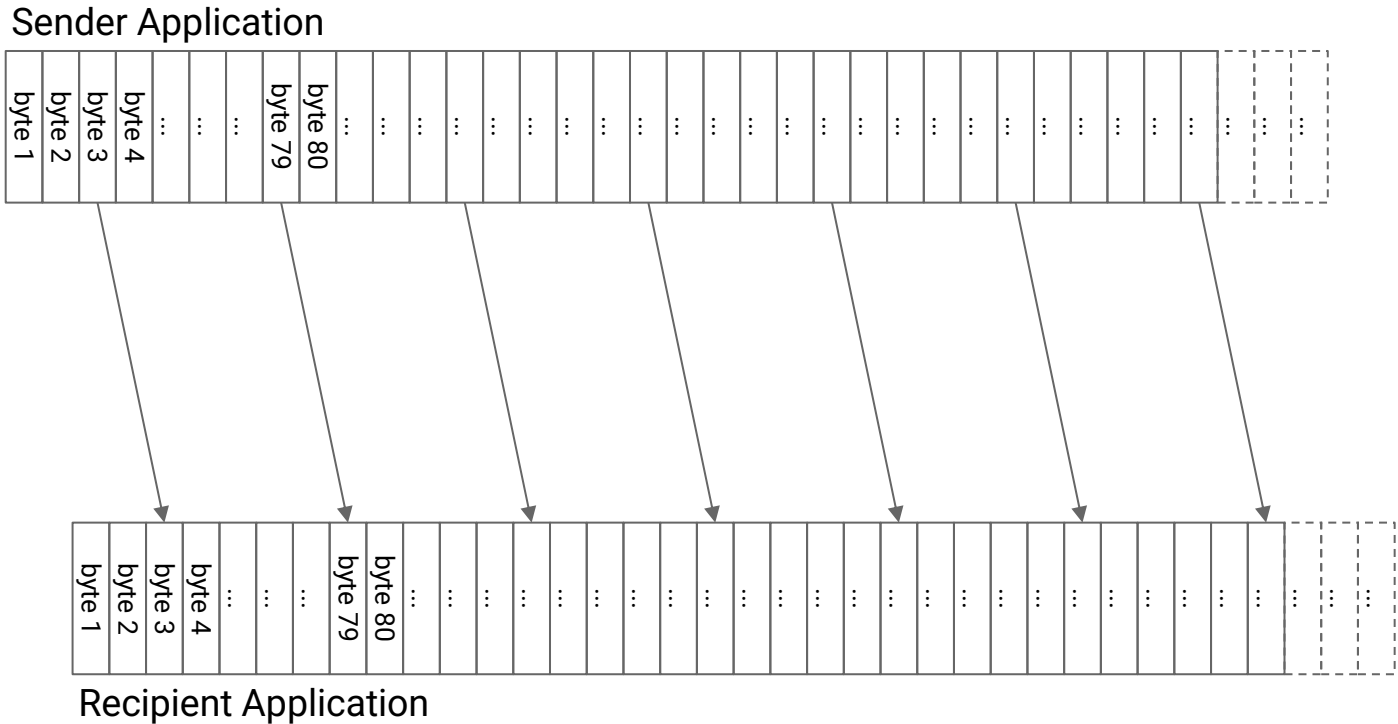
TCP is implemented with *bytes* as the primary unit of data.

- Each byte has a number. Packets are defined by the number of the first byte inside.
- ACKs reference byte numbers.
- Window size expressed in terms of number of bytes.

Lecture 10 designed reliability; Lecture 11 maps those ideas onto real TCP bytes, state, flags, and headers

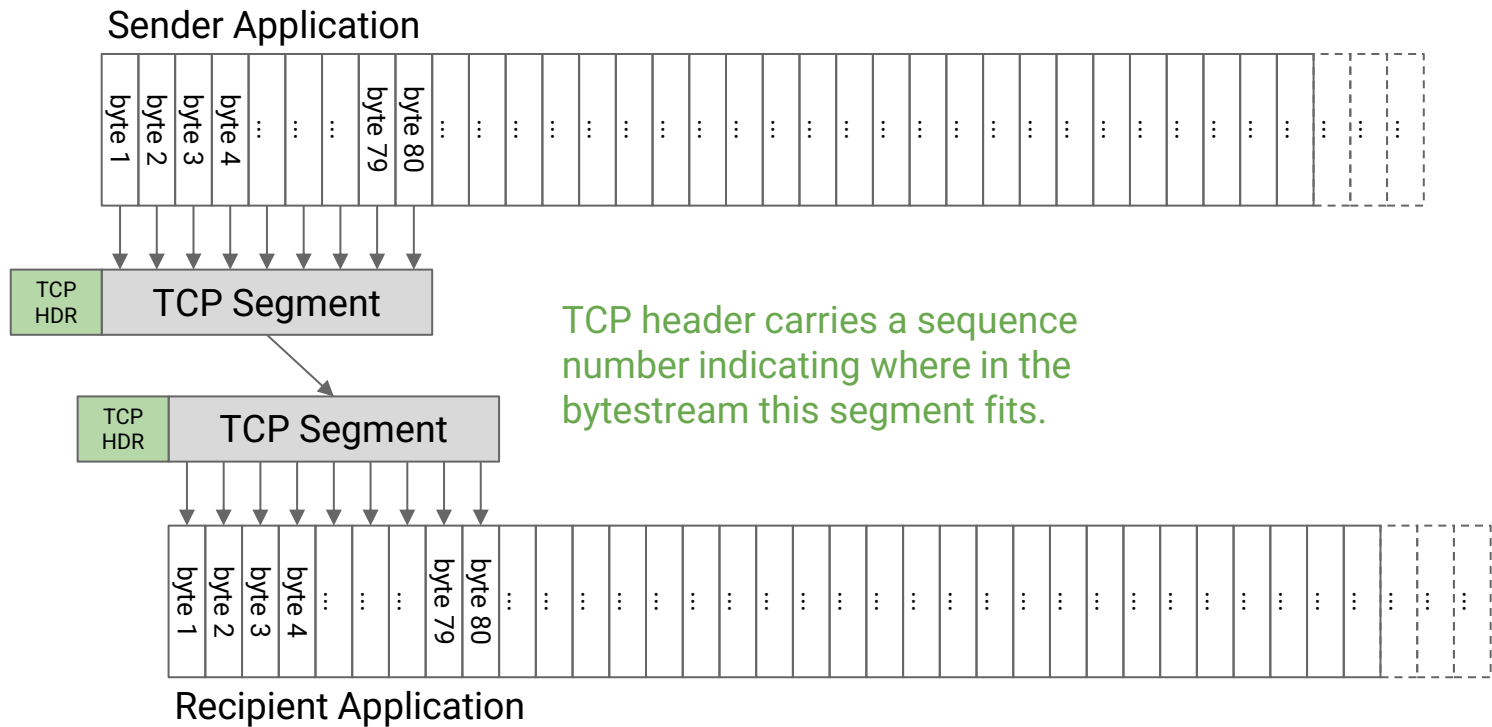
TCP provides a reliable, in-order bytestream.

We have to split this bytestream into packets.



TCP Segments

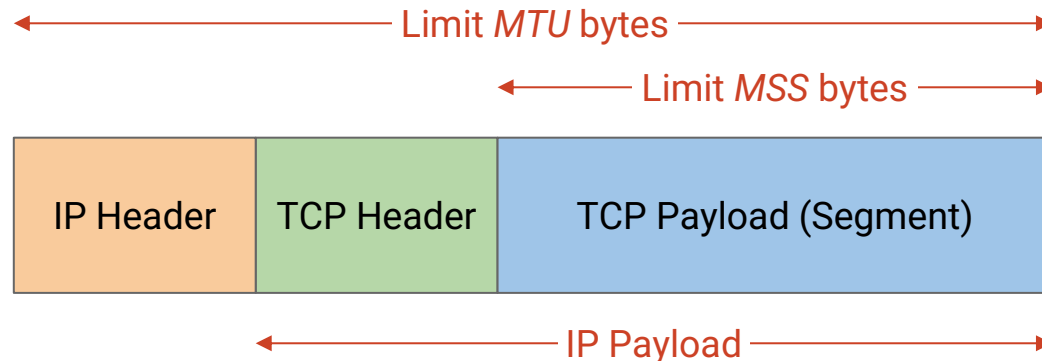
A segment is sent when the segment is full (max segment size),
or when the segment is not full, but times out waiting for more data.



TCP/IP packet: IP packet with TCP header and TCP data inside.

Size limits:

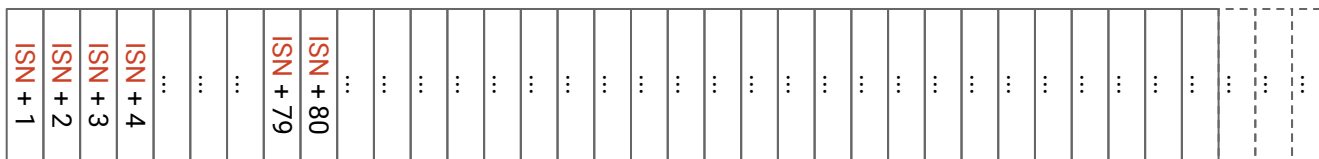
- IP packet: Maximum transmission unit (MTU).
- TCP segment: Maximum segment size (MSS).
- $MSS = MTU - (IP\ header) - (TCP\ header)$.



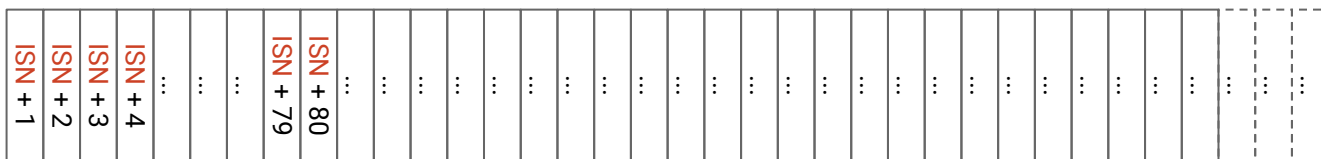
TCP Sequence Numbers

Numbering starts at a randomly-generated **Initial Sequence Number** (ISN).

- First byte is $ISN+1$, then $ISN+2$, etc.
- Random ISNs reduce the risk that old duplicate segments will be mistaken for new data, and mitigate some spoofing attacks (for security).



Sender Application

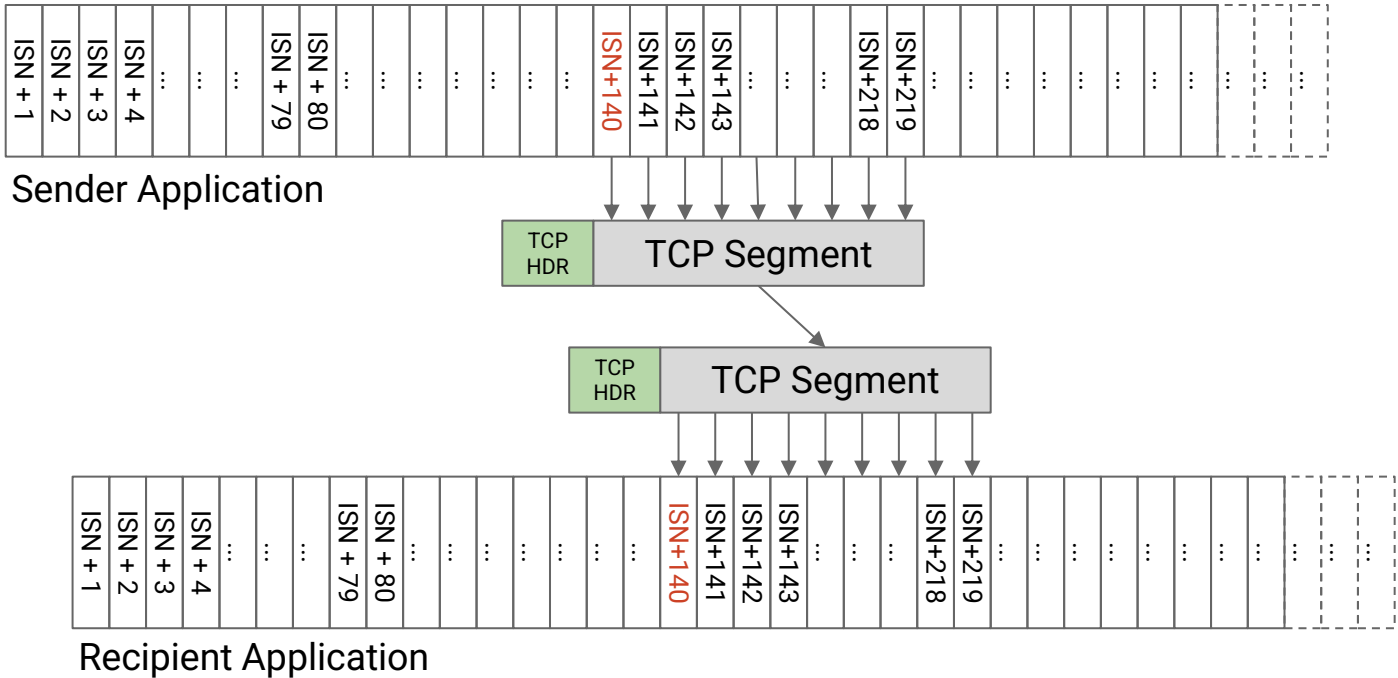


Recipient Application

TCP Sequence Numbers

The sequence number of a segment is the number of the *first byte* in the segment.

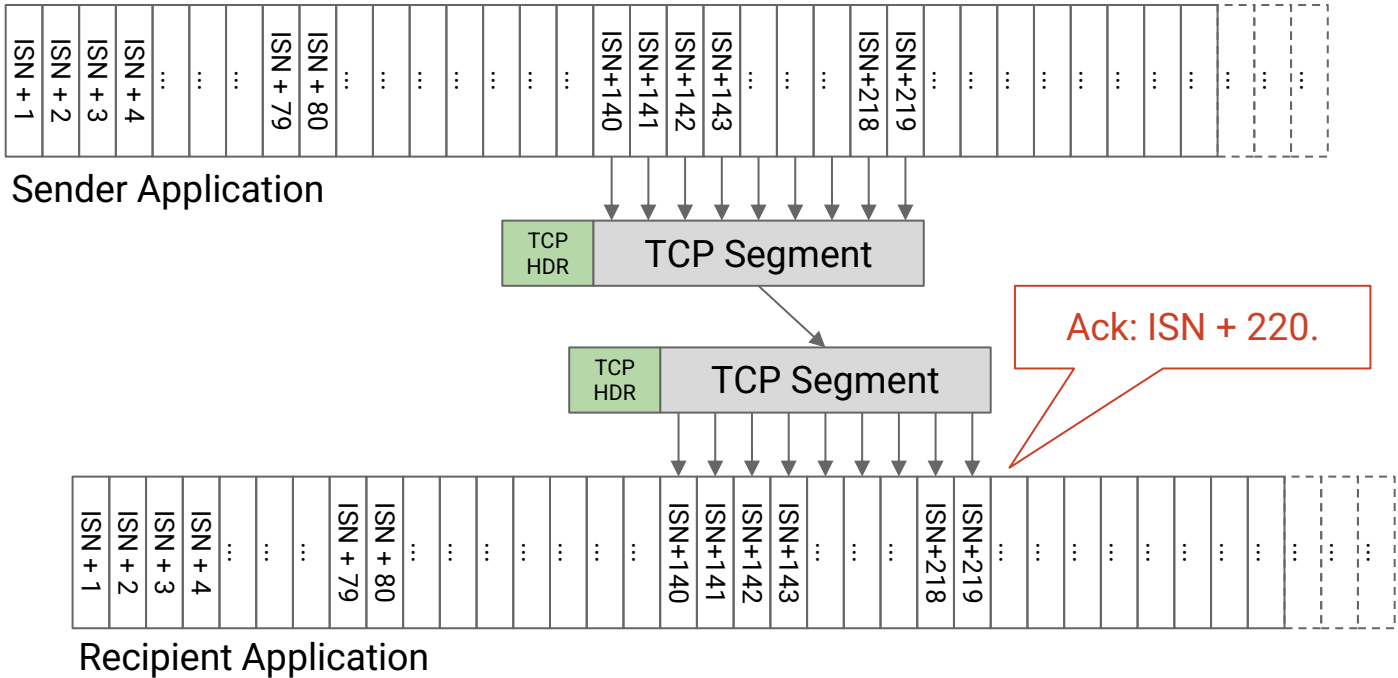
Example: In the segment below, the sequence number is *ISN + 140*.



TCP Ack Numbers

The ack number indicates the next expected byte (i.e. the first unreceived byte).

Example: All bytes up to (and including) *ISN + 219* have been received, so the next unreceived byte is *ISN + 220*.



The sequence number of a segment is the number of the *first byte* in the segment.

- Sender sends a packet with sequence number j .
- The packet contains B bytes.
- bytes in the packet are numbered: $j, j+1, j+2, j+3, \dots, j+B-1$.

Recipient sends a cumulative ack (number of highest byte received, plus one).

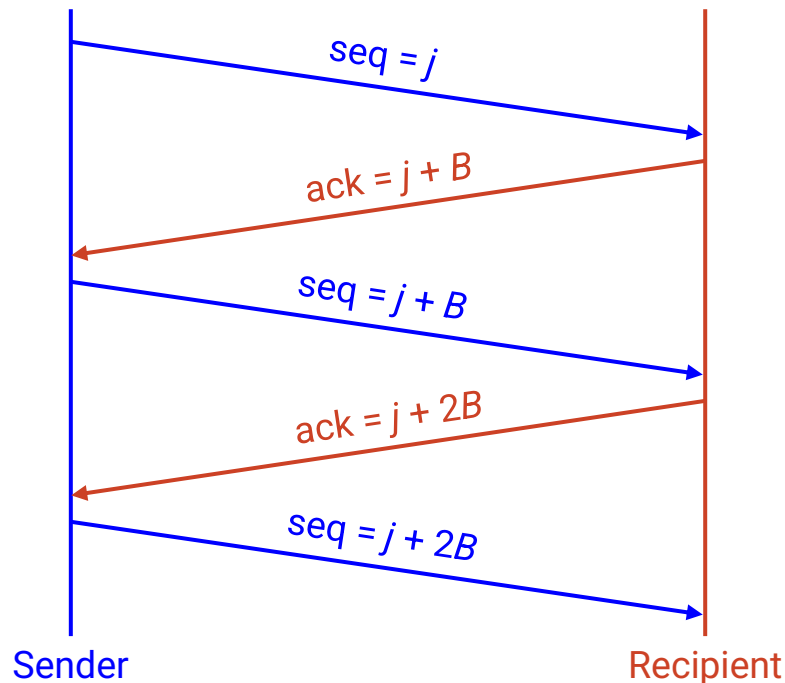
- If all prior data is received, ack number is $j+B$, the next expected byte, or the first unreceived byte.
- If earlier data before this packet is missing, the ack number will be lower.

TCP Sequence and Ack Numbers

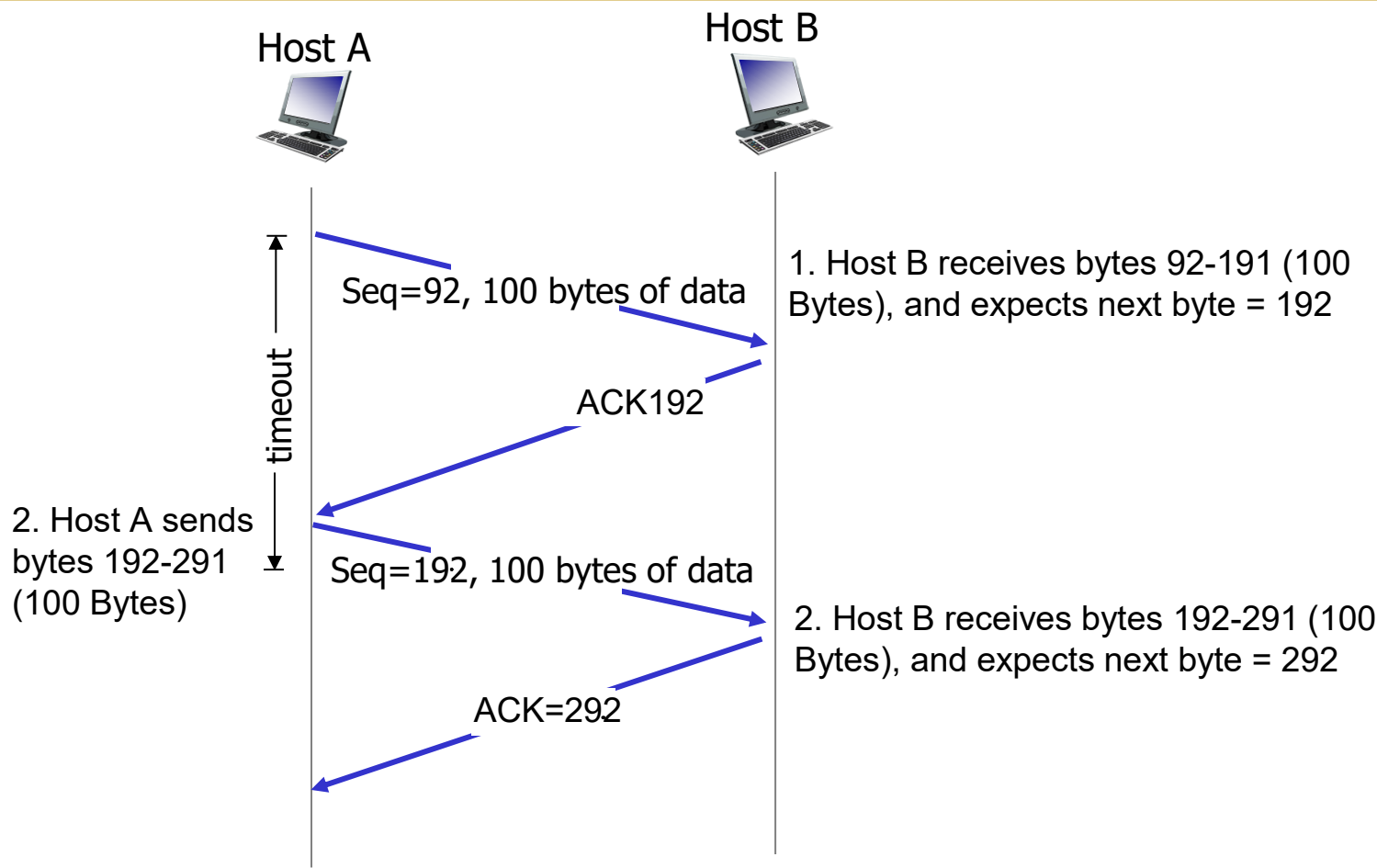
Assuming only one packet in flight, all packets length B , and no loss:

The last ack number is equal to the next sequence number.

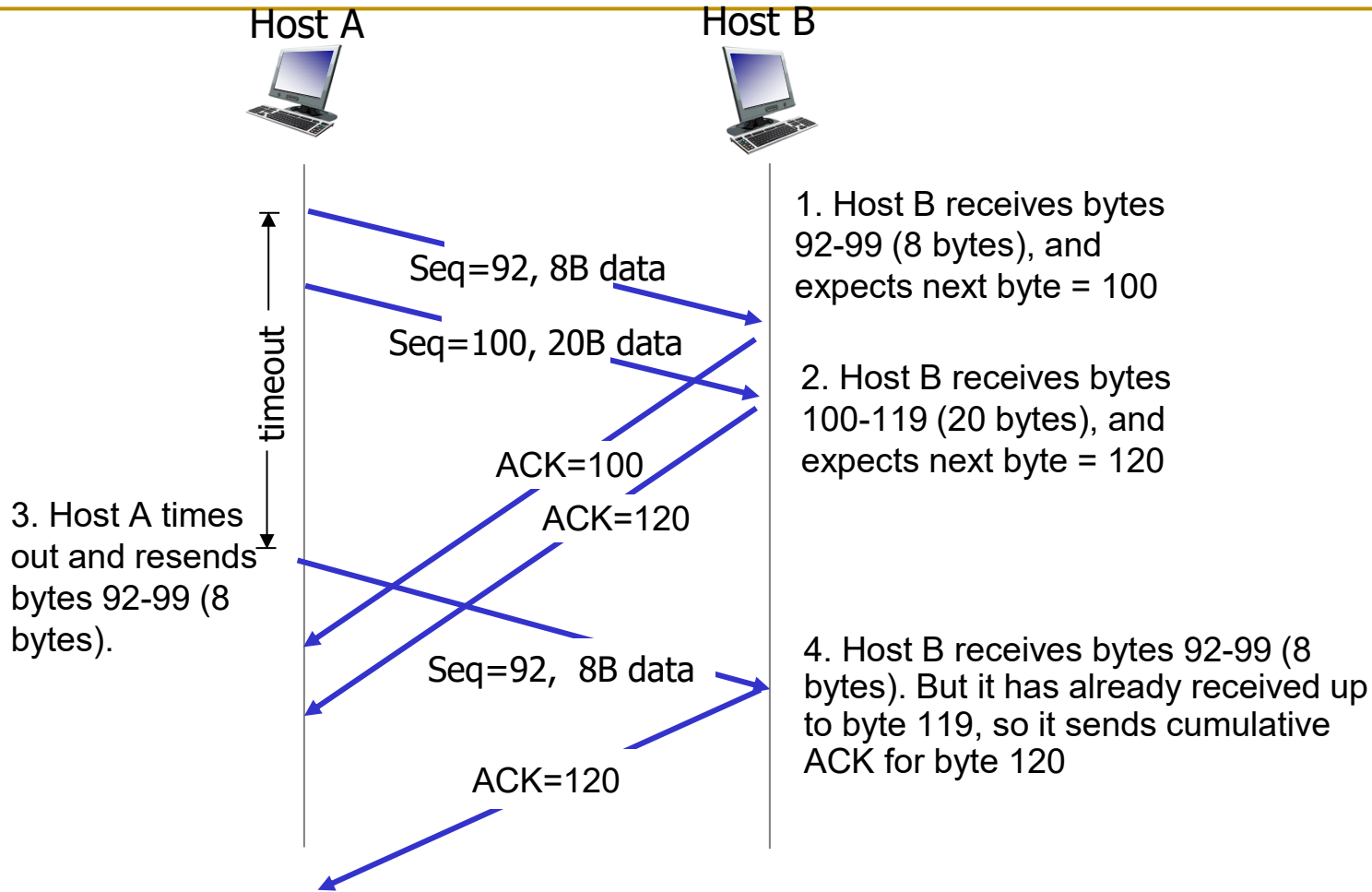
- Receiver ack: "I expect $j + B$ next." → Sender: "I'm sending $j + B$."



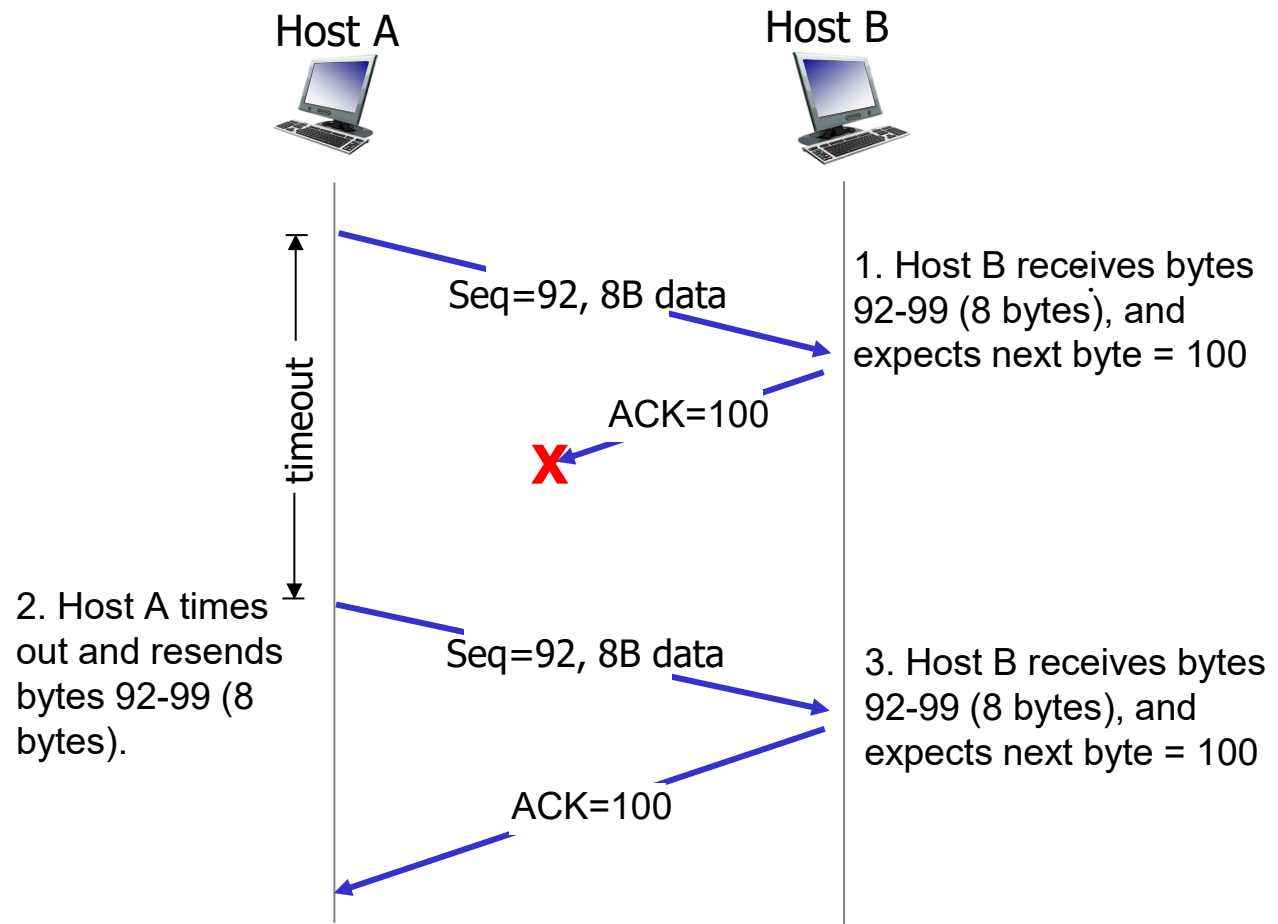
Received ACK Scenario



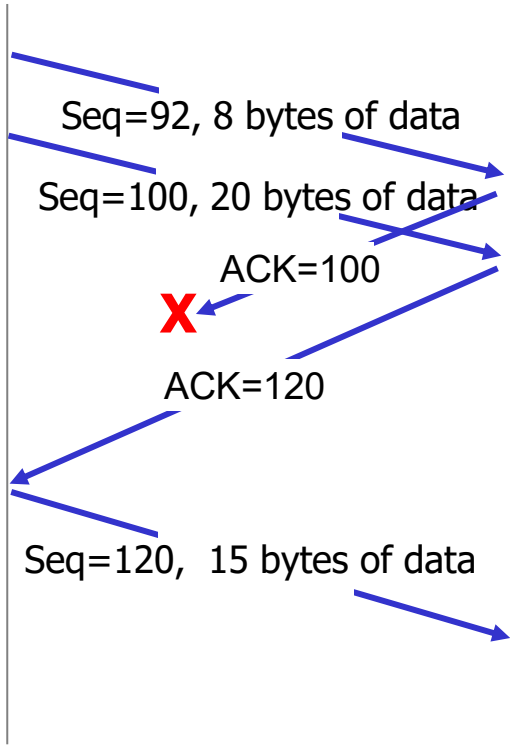
Premature Timeout



Lost ACK Scenario



Lost ACK Scenario 2



- 1. Host B receives bytes 92-99 (8 bytes), and expects next byte = 100, but the ACK is lost
- 2. HostB receives Seq# 100-119 (20 bytes), and expects the next received Seq# to be 120

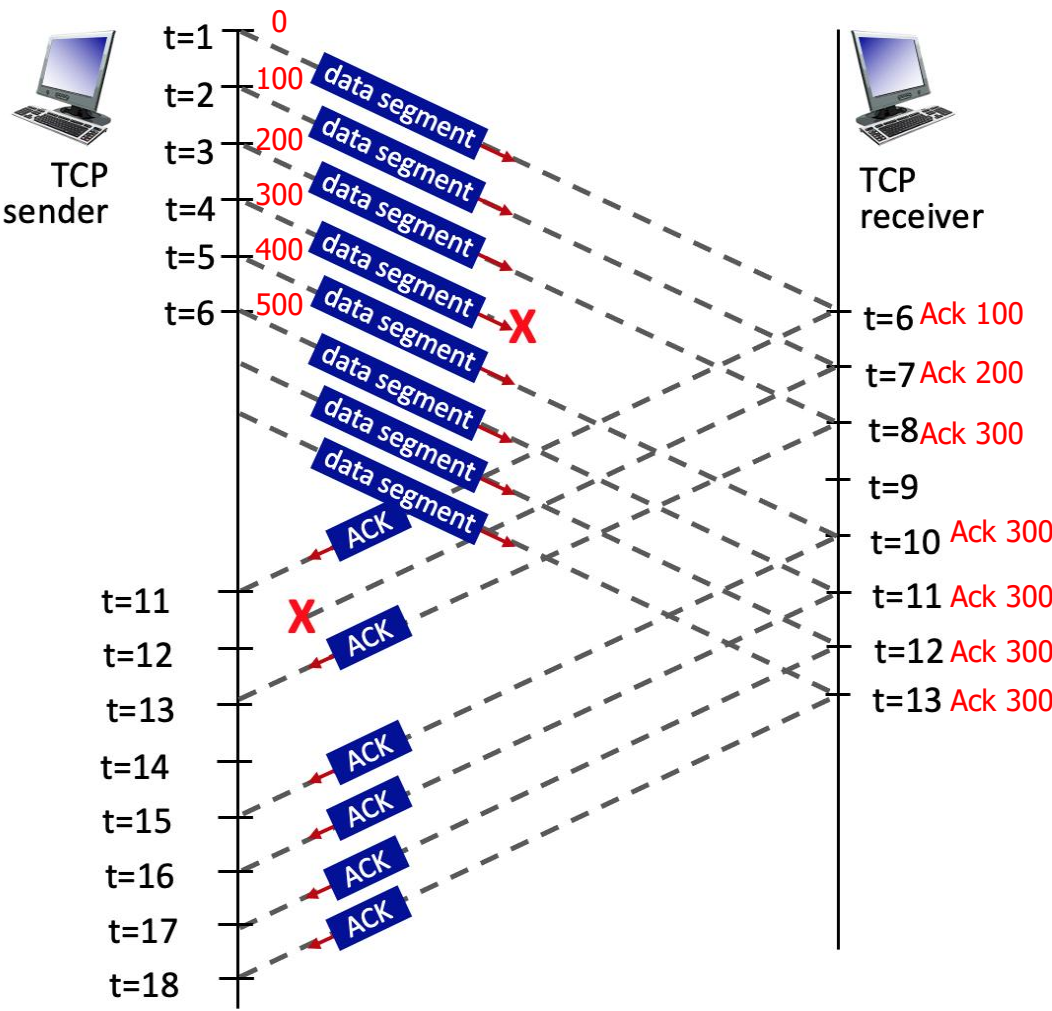
3. Host A receives ACK for byte 120. **This cumulative ACK covers for the earlier lost ACK of byte 100**, so Host A knows that Host B has received all bytes up to 119, so it can send the next bytes 120-134 (15 bytes)

Q: what happens if the segment with Seq=92, 8 bytes of data from Host A to Host B gets lost?

A: Host B will NOT send ACK=120, since a cumulative ACK=120 implies that all previous segments with Seq < 120 have been received. Host B will send ACK=92, assuming all all previous segments with Seq < 92 have been received.

TCP Sequence and ACK Numbers

- Consider the figure, where a TCP sender sends 8 TCP segments at $t = 1, 2, 3, 4, 5, 6, 7, 8$. Suppose the initial value of the sequence number is 0 and every segment sent to the receiver each contains 100 bytes. The delay between the sender and receiver is 5 time units, and so the first segment arrives at the receiver at $t = 6$. The ACKs sent by the receiver at $t = 6, 7, 8, 10, 11, 12$ are shown. The TCP segments (if any sent by the sender at $t = 11, 13, 15, 16, 17, 18$ are not shown. The segment sent at $t=4$ is lost, as is the ACK segment sent at $t=7$.
- Q: What is the sequence number of the segment sent at $t=2$?
- ANS: $0+100=100$.
- Q: What is the ACK value carried in the receiver-to-sender ACK sent at $t = 6$?
- ANS: $0+100=100$. By sending this ACK, the receiver is telling the sender: "I've received everything up to byte 99, and I'm now expecting byte 100."
- Q: What is the ACK value carried in the receiver-to-sender ACK sent at $t = 8$?
- ANS: 300. By sending this ACK, the receiver is telling the sender: "I have received everything up to byte 299. Please send me the next byte, which should be 300."
- Q: What is the ACK value carried in the receiver-to-sender ACK sent at $t = 10$?
- A: 300. By sending this ACK, the receiver is telling the sender: "I have received everything up to byte 299. Please send me the next byte, which should be 300." (Even though the receiver has received segments sent after the lost one, it cannot acknowledge them yet due to cumulative ack mechanism)



TCP sequence and ACK numbers Explanations

- TCP uses sequence numbers to keep track of the bytes sent in a data stream. Each byte of data is assigned a unique sequence number.
- Given Information: Initial sequence number: 0 - Each segment contains 100 bytes. Segments are sent at $t = 1, 2, 3, 4, 5, 6, 7, 8$.
- To find the sequence number of the segment sent at $t=2$, consider the following:
 - 1. The initial sequence number is 0 for the first segment ($t=1$).
 - 2. Each segment contains 100 bytes.
 - 3. The sequence number for each subsequent segment increases by the number of bytes in the previous segment.
- 1. The first segment ($t=1$) starts with sequence number 0. 2. Each segment contains 100 bytes of data. 3. The second segment ($t=2$) will have a sequence number that is the sum of the initial sequence number (0) and the number of bytes in the first segment (100).
 - Segment at $t=1$: Sequence number = 0
 - Segment at $t=2$: Sequence number = $0 + 100 = 100$
 - Segment at $t=3$: Sequence number = $100 + 100 = 200$
 - Segment at $t=4$: Sequence number = $200 + 100 = 300$
- The lost segment at $t=4$ and the lost ACK at $t=7$ do not affect the sequence numbers of the segments being sent. They would, however, impact the retransmission and acknowledgment process in a real TCP connection.

TCP sequence and ACK numbers Explanations

- Given: The initial sequence number is 0. 8 TCP segments are sent at $t = 1, 2, 3, 4, 5, 6, 7, 8$. Each segment contains 100 bytes. The delay between sender and receiver is 5 time units. The segment sent at $t=4$ is lost. We need to determine the ACK value sent by the receiver at $t = 6$.
 - Segments Received: At $t = 6$, only the first segment (sent at $t = 1$) has arrived at the receiver due to the 5-unit delay.
 - bytes Received: The first segment contains 100 bytes.
 - ACK Mechanism: TCP uses cumulative ACKs, which means it acknowledges all bytes up to the last contiguously received byte.
 - ACK Value Calculation: Initial sequence number: 0; bytes successfully received: 0-99 (100 bytes); Next expected byte: $0 + 100 = 100$
- Therefore, the ACK value carried in the receiver-to-sender ACK sent at $t = 6$ is 100, which indicates that the receiver has successfully received all bytes up to, but not including, byte 100. It's essentially saying, "I've received everything up to byte 99, and I'm now expecting byte 100."
- Even though more segments were sent by the sender, due to the network delay, only the first segment has reached the receiver at $t = 6$. The lost segment at $t = 4$ doesn't affect this initial ACK because it hasn't even reached the receiver yet.

Maintaining State (Full Duplex)

Lecture 11, Spring 2026

Implementing TCP

- byte Notation
(Segments, Sequence Numbers)
- **Maintaining State (Full Duplex,
Connection Setup and Teardown)**
- Sliding Window
- Header

Reliability requires maintaining *state* at the end hosts.

- Sender has to remember:
 - Which packets have been sent and not acked?
 - How much longer on the timer before I resend a packet?
- Receiver has to buffer the out-of-order packets.
- State is maintained at the end hosts, not in the network.

In each separate connection, both end hosts need to maintain state.

TCP connections are **full-duplex**.

- Both hosts can send and receive data simultaneously in the same connection.
- Two independent sequence-number spaces: One for A→B bytes, and one for B→A bytes.
- Each packet carries both data and ack information.
 - "Here are some bytes starting at 15..."
 - "...Also, I ack receiving all your bytes up to (not including) 84."

15	16	17	18	19	20	21
H	e	l	l	o	,	B

A to B bytestream

77	78	79	80	81	82	83
H	e	l	l	o	,	A

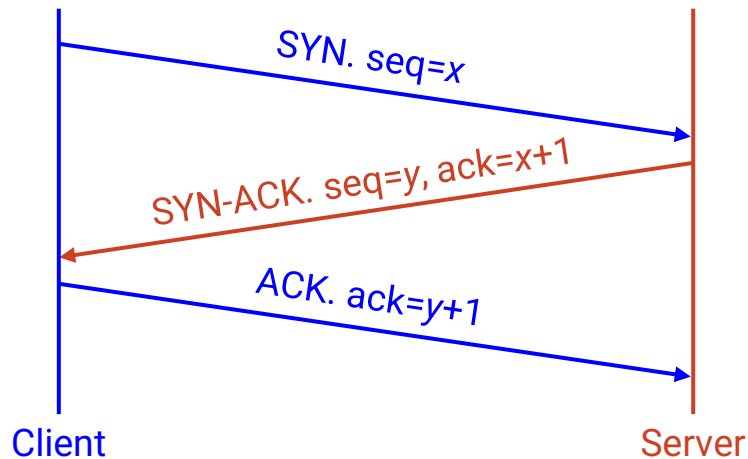
B to A bytestream

TCP Connection Setup: Three-Way Handshake

Goal: Each host tells its ISN to the other host.

1. **SYN**. Client says: "My ISN is x ."
2. **SYN-ACK**. Server says: "I received x (expecting $x+1$ next). Also, my ISN is y ."
3. **ACK**. Client says: "I received y (expecting $y+1$ next)."

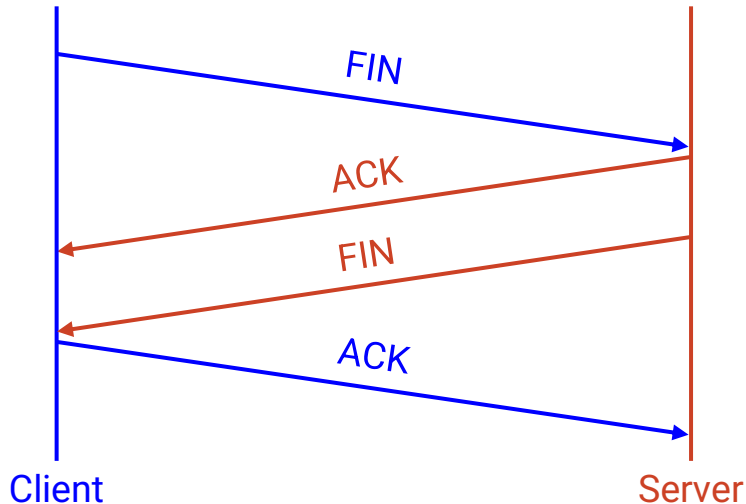
After the three-way handshake, both sides can start sending data.



TCP Connection Teardown

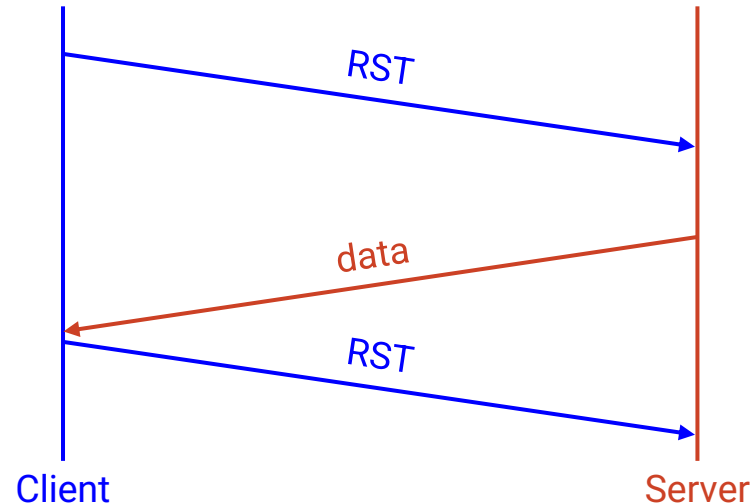
Normal termination with FIN:

- Each side sends a FIN packet to say: "I'm done sending, but will keep receiving."
- FIN packets are acked like any other data.
- When only one side has sent FIN, the connection is *half-closed*.
- When both sides have sent FIN (both done sending), the connection is closed.

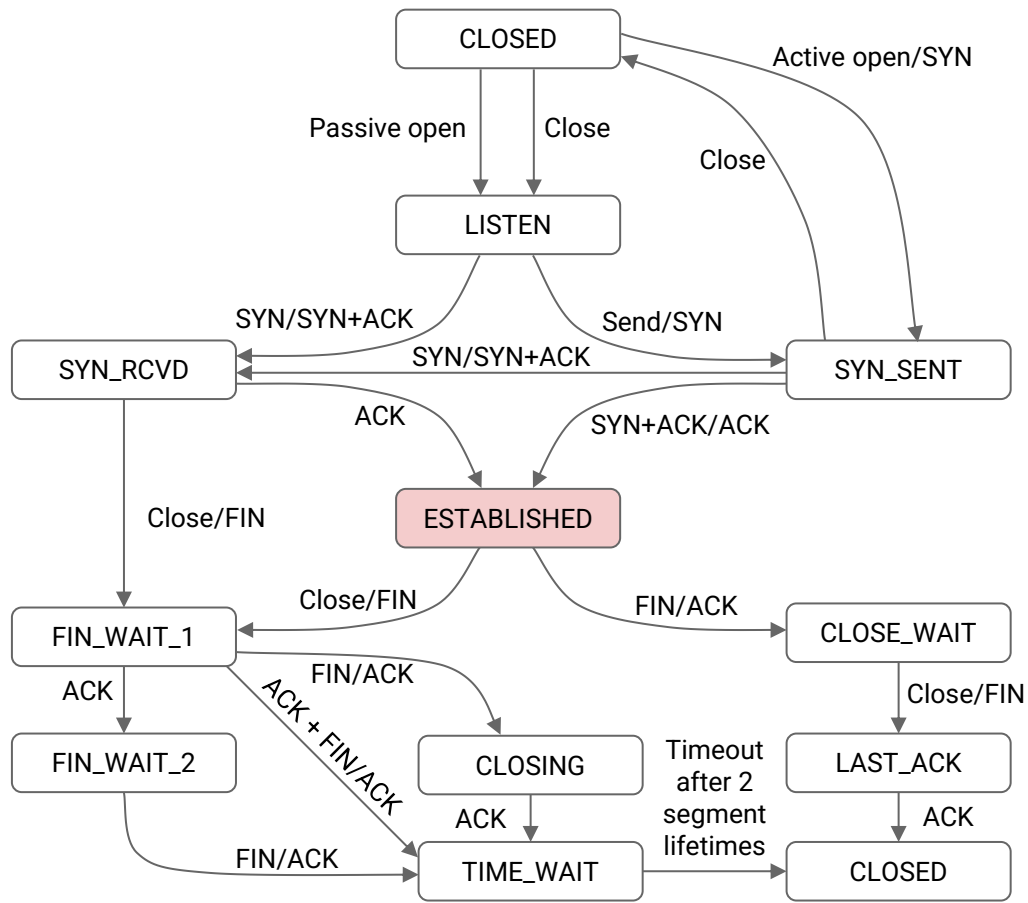


Abrupt termination with RST:

- Send a RST (reset) to say: "I will no longer send or receive data."
- RST packets do not need to be acked.
- Any data in flight is lost.
- If the RST sender receives more data later, send another RST.

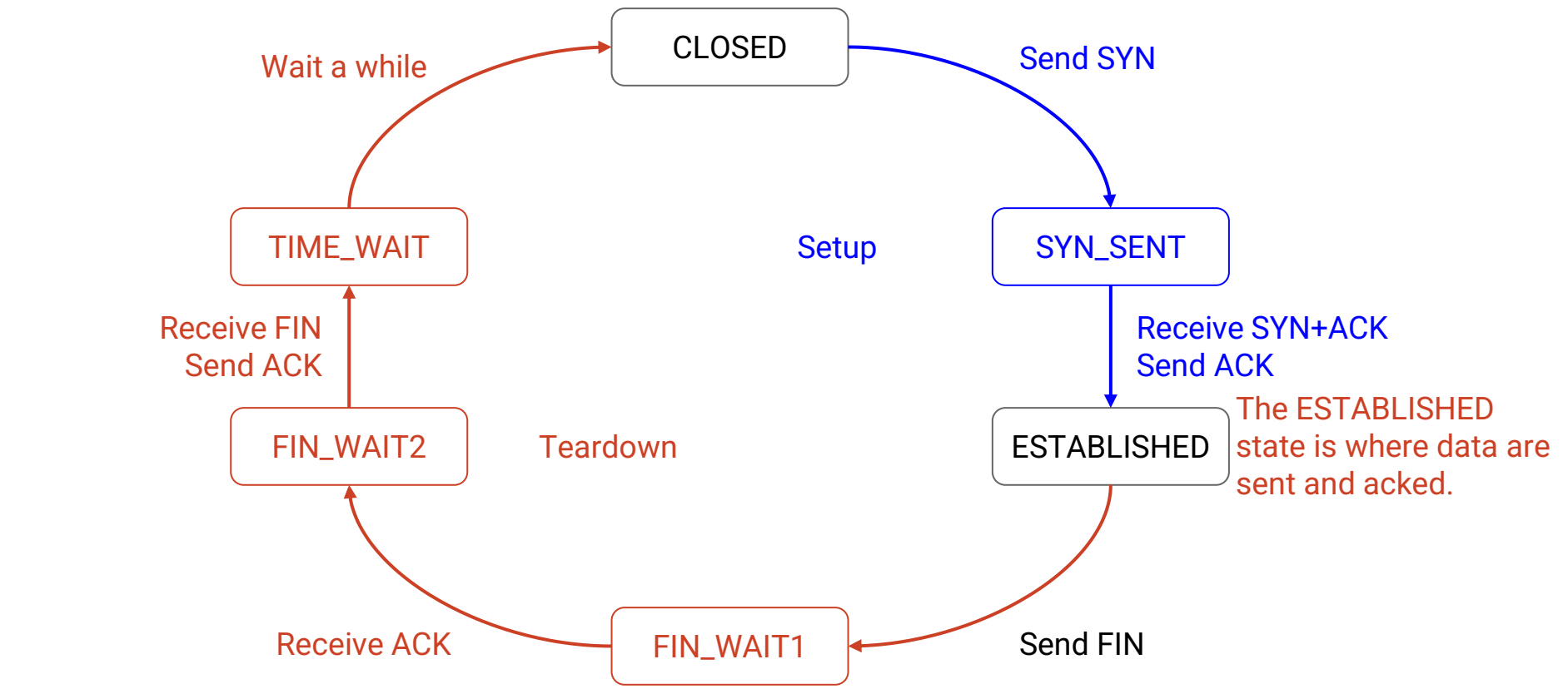


TCP Setup/Teardown Transition Diagram



The **ESTABLISHED** state is where data are sent and acked.

TCP Setup/Teardown Transition Diagram (Simplified)



With full-duplex, if we get a packet but have no data to send, we have two choices:

- Send the ack, with no data.
- **Piggybacking**: Wait for some data, and send the ack with the data.

Piggybacking can be tricky because TCP is in the OS, separate from the application.

- OS doesn't know when application will have more data.
- Application isn't thinking about packets and acks.

SYN-ACKs are always piggybacked.

- Ack and initial sequence number are sent together.
- Not tricky, because OS is doing the handshake, not the application.

Sliding Window

Lecture 11, Spring 2026

Implementing TCP

- byte Notation
(Segments, Sequence Numbers)
- Maintaining State (Full Duplex,
Connection Setup and Teardown)
- **Sliding Window**
- Header

TCP Sliding Window

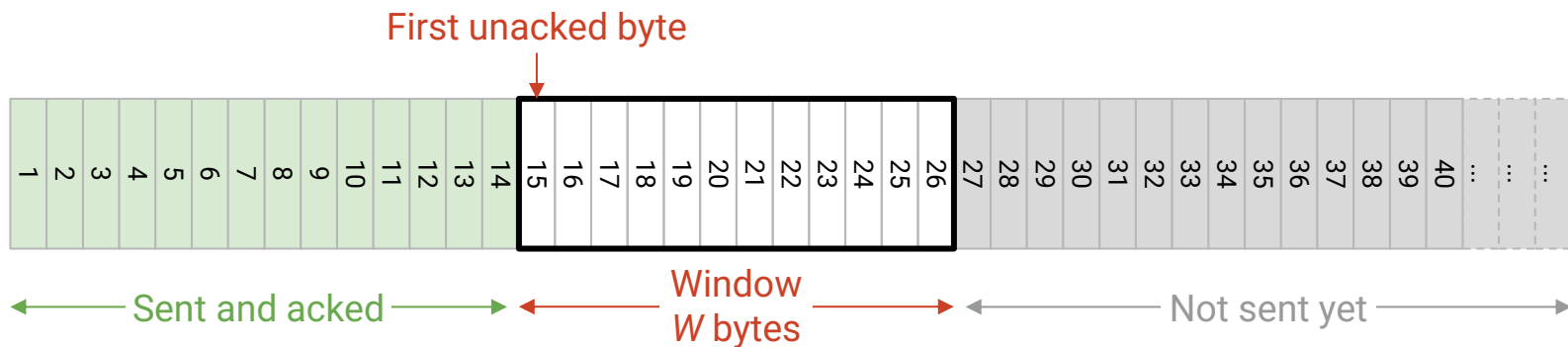
When we measured in packets, W was the maximum number of packets in flight.

When we measure in bytes, W is the maximum number of *contiguous* bytes in flight.

- The window is a range of W contiguous bytes, starting at the first unacked byte.
- Only these W bytes are allowed to be in flight.

The window slides right if and only if its *leftmost* bytes are acked.

- Example: When 15–18 arrive, we can send 27–30.



TCP Sliding Window

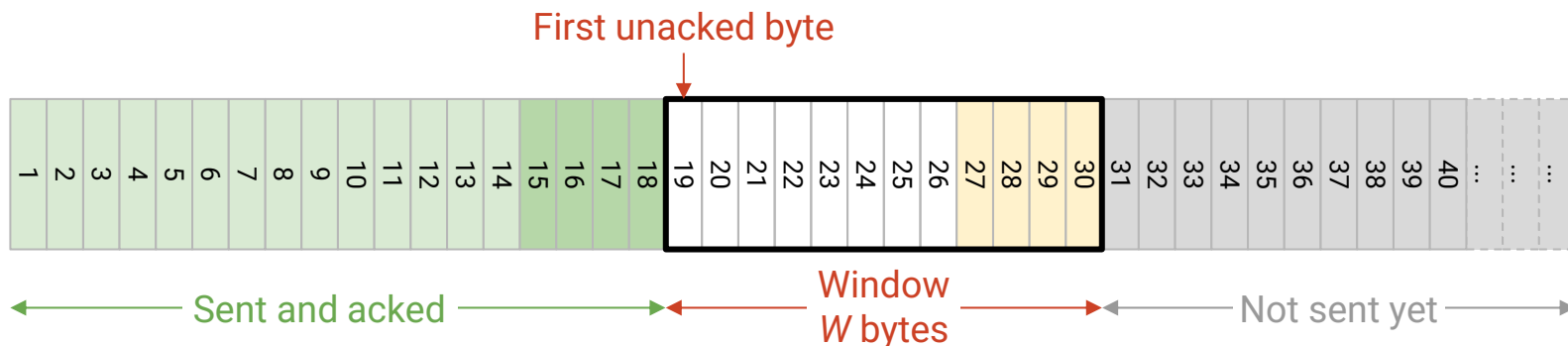
When we measured in packets, W was the maximum number of packets in flight.

When we measure in bytes, W is the maximum number of *contiguous* bytes in flight.

- The window is a range of W contiguous bytes, starting at the first unacked byte.
- Only these W bytes are allowed to be in flight.

The window slides right if and only if its *leftmost* bytes are acked.

- Example: When 15–18 arrive, we can send 27–30.



TCP Sliding Window

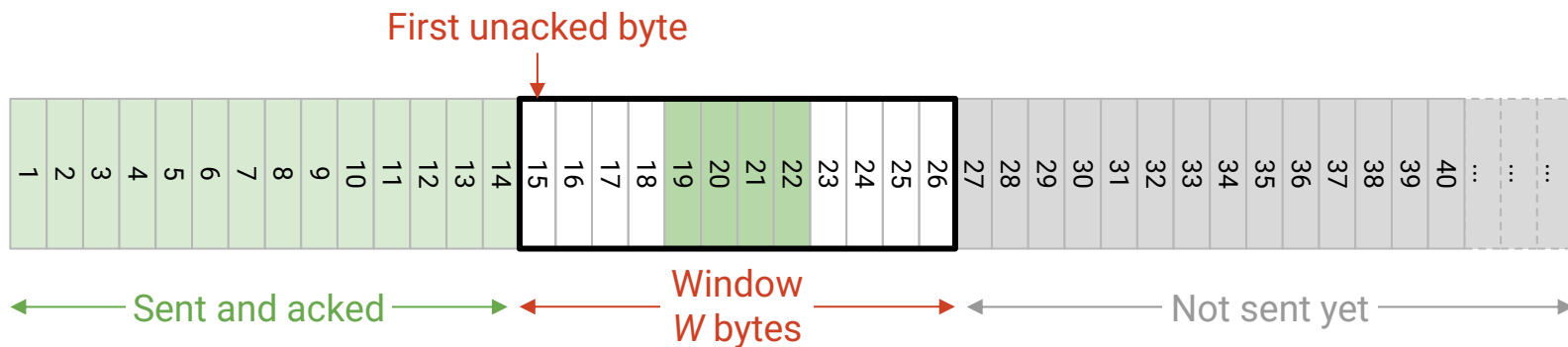
When we measured in packets, W was the maximum number of packets in flight.

When we measure in bytes, W is the maximum number of *contiguous* bytes in flight.

- The window is a range of W contiguous bytes, starting at the first unacked byte.
- Only these W bytes are allowed to be in flight.

The window slides right if and only if its *leftmost* bytes are acked.

- Acking non-leftmost bytes in the window (e.g. 19–22) does not slide the window.
- The window is determined by the first unacked byte (e.g. still 15).



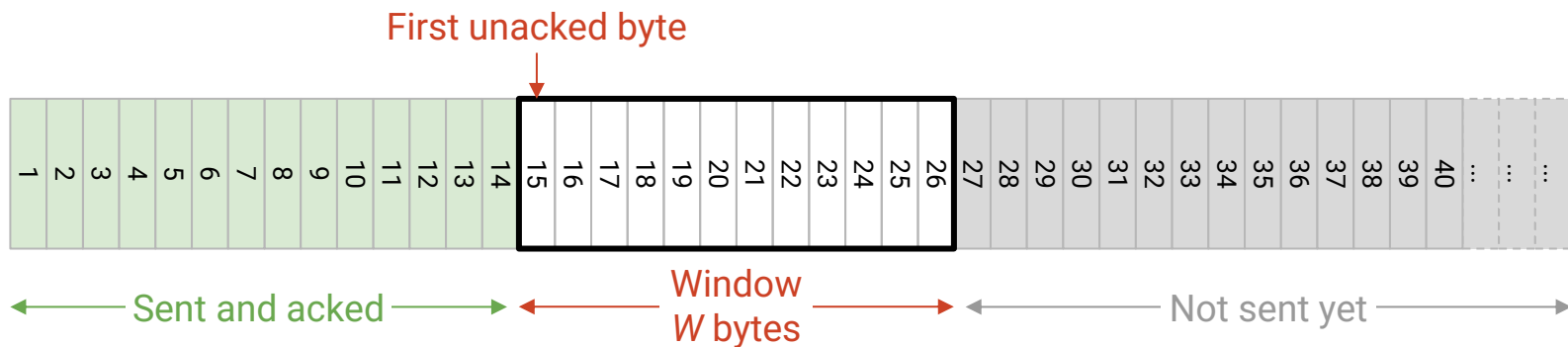
TCP Sliding Window

TCP uses cumulative acks and a sliding window.

- Cumulative ack: "I have received everything up to (not including) 15."
- Thus, first unacked byte is 15, and the sliding window is $[15, 15 + W)$.

W is set as the minimum of two values:

- Advertised window (recipient reports their remaining buffer space).
- Congestion window (sender magically finds value to avoid network overload).

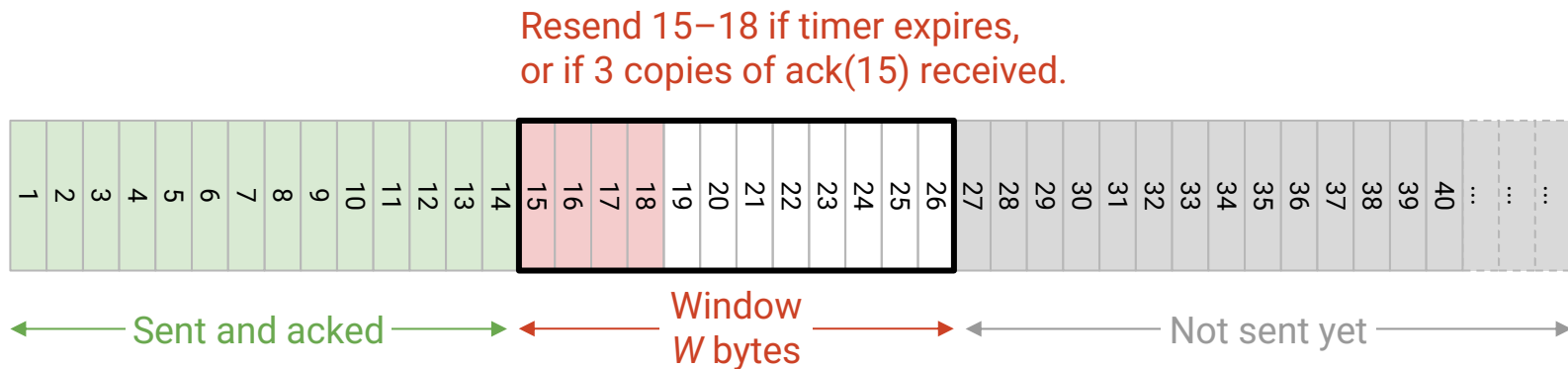


Detecting Loss

Two ways to detect loss and resend. We resend if either condition is true.

In both cases, we always resend the *first unacked packet* (leftmost part of window).

1. Ack-based: If 3 duplicate acks are received, resend.
 - Receiving three copies of the same cumulative ACK signals that a later segment arrived but an earlier one is missing, so the sender immediately retransmits the first unacknowledged segment (leftmost byte of the window) without waiting for the timer.
2. Timer-based: Keep a single timer. If the timer expires, resend.
 - Set timer by estimating RTT (e.g. measure times between packets and acks, and take a moving average).



TCP Header

Lecture 11, Spring 2026

Implementing TCP

- byte Notation
(Segments, Sequence Numbers)
- Maintaining State (Full Duplex,
Connection Setup and Teardown)
- Sliding Window
- **Header**

What functions does TCP implement?

1. Demultiplexing (*ports*)
2. Reliability (*checksum, sequence and ack numbers*)
3. Connection setup and teardown (*flags*)
4. Flow control (*advertised window*)

Source Port (16)			Destination Port (16)		
Sequence Number (32)					
Acknowledgment Number (32)					
Hdr Len (4)	0000	Flags (8)		Advertised Window (16)	
Checksum (16)			Urgent Pointer (16)		
Options (variable-length)					
Payload					

There are 8 flags that can be set in TCP. We care about 4 of them:

- SYN: I'm sending my initial sequence number.
- ACK: I'm acking data (please look at the ack number).
- FIN: I'm done sending data, but will keep receiving data.
- RST: I'm done sending and receiving data.

We won't look at the other four: CWR, ECE, URG, PSH.

Source Port (16)			Destination Port (16)		
Sequence Number (32)					
Acknowledgment Number (32)					
Hdr Len (4)	0000	Flags (8)		Advertised Window (16)	
Checksum (16)			Urgent Pointer (16)		
Options (variable-length)					
Payload					

Remaining fields:

- Header length: Measured in 4-byte words.
 - If no options, this is 5 (header is 20 bytes long).
- 0000: Reserved bits (always set to 0).
- Urgent pointer: Used with the URG flag to indicate urgent data. Won't discuss.
- Options: Extra functionality.
 - Example: SACK uses the options field to implement full-information acks.

Source Port (16)			Destination Port (16)		
Sequence Number (32)					
Acknowledgment Number (32)					
Hdr Len (4)	0000	Flags (8)		Advertised Window (16)	
Checksum (16)			Urgent Pointer (16)		
Options (variable-length)					
Payload					

An elegant (though not perfect) piece of engineering that has stood the test of time.

- Thought experiment: Will TCP continue to be a good solution?

Plenty of evolution in individual pieces:

- Congestion control was added after-the-fact.
- Better acknowledgments, ISN selection, timer estimation, etc.

But the core architectural decisions and abstractions remain:

- bytestreams, connection-oriented, windows, etc.